

# 面向 AI 原生系统的智能运维

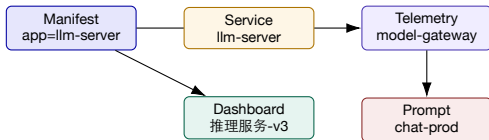
## 第 15 讲 | 运维对象档案、实体对齐与关联构建

---

先回答“世界里有哪些对象”，再回答“它们为什么发生了什么”

陈鹏飞 | 中山大学计算机学院  
校企联合研究生课程 2026 秋

## 开场问题：同一个服务，为什么会有五个名字？



- ▶ 文本抽取只能告诉我们“出现了哪些短语”；
- ▶ 对象档案还要回答：是不是同一个对象？哪个版本？谁负责？何时有效？
- ▶ 关系图要把声明配置、运行时观测、文档和人工记录放到同一语义空间。

**本讲主线** 从多源材料中生成可追溯、可更新、可被后续 RCA 和流程编排消费的对象图。

---

# 课程大纲中的本讲任务

## 大纲要求

围绕主机、服务、应用、接口、Pod 等运维对象，从配置、文档、告警、日志和人工记录中抽取对象信息，形成对象档案，并建立对象之间、对象与文档之间、对象与事件之间的关联关系。

输入 配置、文档、日志、告警、人工记录、Telemetry。

产物 对象档案、别名表、关系边、来源和时间。

下游 第 16 讲事件记忆、第 17 讲诊断 Agent、第 14 讲流程编排。

---

## 先划边界：本讲不重复什么？

---

| 章节     | 主要问题            |                                                 |
|--------|-----------------|-------------------------------------------------|
| 第 12 讲 | AI 系统与模型服务如何运行？ | 第 15 讲的边界<br>复用模型、端点、Pod 等对象名，不重讲推理运行时          |
| 第 13 讲 | 哪条证据链支持 RCA？    | 只准备稳定对象和关系，不做根因排序                               |
| 第 14 讲 | 请求如何变成安全流程？     | 第 14 讲消费 canonical object；本讲创建 canonical object |
| 第 16 讲 | 告警如何合并为对象历史？    | 本讲提供对象 ID 和关系，事件记忆留给下一讲                         |
| 第 17 讲 | Agent 如何分析并修复？  | 本讲不展开调查循环、修复和回滚                                 |

---

**边界判断** 本讲的评价标准不是“能不能诊断”，而是“对象是否可识别、关系是否有证据、状态是否可追溯”。

---

## 学习目标：从文本到可计算对象图

1. 认识 AI Native 系统中的对象家族与对象生命周期；
2. 设计 schema-first 的多源对象抽取流程；
3. 用 blocking、候选生成、LLM verifier 和人工复核完成实体对齐；
4. 建立声明、控制面、运行时、文档、血缘和 profiling 关系；
5. 给每个属性和关系附加来源、置信度、有效时间与过期策略；
6. 读懂 UniversalNER、Jellyfish、iText2KG 等论文的结果与边界；
7. 从一个 LLM Deployment 清单构建可供第 16—17 讲消费的对象档案。

关键词：identity、profile、relation、provenance、valid time、freshness。

## 2 学时路线图：对象先于事件，身份先于关系



- ▶ 前半段回答“有哪些对象、如何确认同一性”；
- ▶ 中段回答“哪些关系是声明的、哪些关系由运行时观测得到”；
- ▶ 后半段回答“如何把档案落到一个真实的模型部署案例”。

# 一、多智能体系统基础

先理解 Agent、协同和 Memory，再讨论它们如何进入运维对象图

---

## 为什么 AI Native AIOps 会走向多智能体？

**任务异构** 配置解析、网络探测、GPU profiling、文档检索各自需要不同工具与提示。

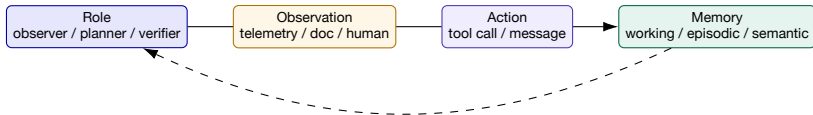
**证据分散** 一个 Agent 很难同时掌握控制面、数据面、模型和组织知识。

**风险分层** 观察、解释、建议、执行和审批需要不同权限与责任。

- ▶ 多智能体不是“多开几个聊天窗口”，而是对角色、工具、状态和责任进行结构化分工；
- ▶ 系统的核心问题从“模型够不够强”转向“谁负责什么、信息如何传递、结论如何验证”。
- ▶ 对象图要记录 agent、run、message、tool、memory 和 policy 的关系。



## 多智能体的最小定义：角色、观测、动作和记忆



- ▶ Agent 的“身份”由 role、policy、model、tool set 和 memory scope 共同定义；
- ▶ 一次运行必须有 run\_id，消息必须有 sender、receiver、causation\_id 和 evidence；
- ▶ 没有这些字段，后续无法解释“哪个 Agent 在什么上下文下做了决定”。

## 常见多智能体架构：集中、分层、对等、黑板

### Supervisor

- ▶ 中央调度者分派任务
- ▶ 易治理、易形成单点

### Hierarchical

- ▶ 领域 Agent 向上汇报
- ▶ 适合组织和权限层级

### Peer-to-peer

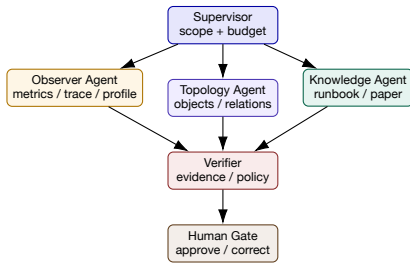
- ▶ Agent 直接协商
- ▶ 灵活但容易形成循环

### Blackboard

- ▶ 共享证据和状态
- ▶ 需要并发与版本控制

运维选择 高风险变更通常需要 Supervisor/Hierarchical；跨域证据汇聚适合 Blackboard；Peer-to-peer 必须设置预算、超时和终止条件。

# 一个 AI Native AIOps 多智能体架构



本讲只使用这张架构图说明对象边界；具体 RCA 调查循环和 Workflow DSL 已在其他章节展开。

# Agent 间消息：自然语言不是可靠的接口

|                     |                        |            |
|---------------------|------------------------|------------|
| 字段                  | 示例                     | 为什么需要      |
| message_id / run_id | m-17 / run-42          | 去重、回放和审计   |
| sender / receiver   | observer / verifier    | 权限和责任边界    |
| intent              | report_evidence        | 明确消息目的     |
| payload schema      | object、relation、sample | 让接收方可验证    |
| causation_id        | span-abc / message-12  | 建立调用链和因果顺序 |
| ttl / budget        | 30s / 4 calls          | 防止无界协商     |
| provenance          | URI、trace、profile、人工记录 | 让结论可以回到证据  |

工程结论 Agent 之间传递的是带 schema 的 Evidence Bundle，而不是一段无法校验的长文本。

# Memory 的五层：短期上下文不等于长期知识



- ▶ Working memory 随 run 消失；Episodic memory 保存一次执行的上下文和结果；
- ▶ Semantic memory 记录可复用对象与关系；Procedural memory 记录如何做；
- ▶ Profile Store 是本讲重点：它必须带版本、时间、来源和状态，而不是简单向量库。

## 多智能体 Memory 的一致性问题的

**并发写入** 两个 Agent 同时修改 owner 或关系边。

**陈旧读取** Verifier 读取到旧的模型版本或已删除 Pod。

**错误固化** 一次错误推断被写入长期语义记忆。

- ▶ 采用 append-only evidence、版本号、optimistic concurrency 和人工确认；
- ▶ 将 memory write 与 memory read 分开授权：观察 Agent 可建议，Verifier 或人工才可提升置信度；
- ▶ 记录 memory\_source、valid\_time、observed\_time、expiry 和 supersedes。

## 本讲的多智能体边界：先建对象，再建事件，再做诊断



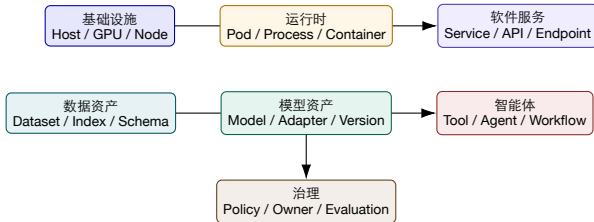
**教学边界** Agent 协同解释“谁可以做什么”；对象档案解释“世界里有什么”；事件记忆和 RCA 再回答“发生了什么、下一步做什么”。

## 二、AI Native 系统中的对象宇宙

对象不是一张 CMDB 表，而是模型、数据、运行时、智能体和治理共同组成的世界



## 对象视角：把 AI 系统拆成可观察的“名词”



对象的边界由身份和生命周期定义；“某个指标”或“某条日志”通常是对象的观测或证据，而不是对象本身。

## 基础设施对象：Host、Node、GPU 与资源池

### Host

- ▶ 机器身份、区域、机架
- ▶ OS、内核、硬件
- ▶ 主机生命周期

### Node

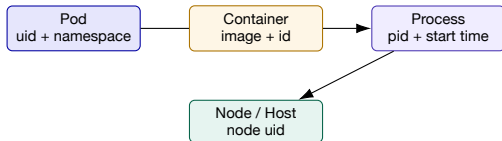
- ▶ Kubernetes 调度单元
- ▶ labels、taints、capacity
- ▶ 与 Pod 的调度关系

### GPU

- ▶ device、型号、显存
- ▶ MIG/partition、健康状态
- ▶ 与进程、Pod 的归属

**教学提示** Host 是物理身份，Node 是编排身份，GPU 是可分配资源；三者不能因为名字相似而合并。

## 运行时对象：Pod、Container、Process



- ▶ Pod 重建后 name 可能不变或相似，但 UID、容器 ID、进程启动时间会变化；
- ▶ 一个长期服务对应多个短生命周期 Pod；关系边要记录有效时间；
- ▶ 运行时对象是 profiling 和 trace 归因的落点。

来源：Kubernetes ownerReferences；OpenTelemetry Entities 草案。

# 软件对象：Component、API、Service、Endpoint



如何读图：

- ▶ Component 是可部署或可维护的软件单元；
- ▶ API 是契约，不等于某一个进程；
- ▶ Resource 是数据库、队列、集群等运行基础设施；
- ▶ System/Domain 提供组织层次。

Backstage 的价值是把“服务目录”变成稳定的语义入口；本课程再将其与运行时和模型对象关联。

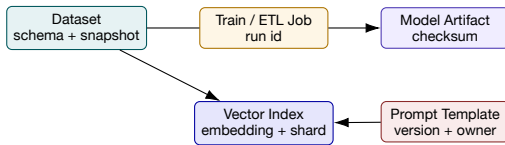
来源：Backstage Software Catalog System Model，官方图，访问 2026-08。

## 模型对象：artifact、version、adapter 与 endpoint

| 对象                        | 稳定身份示例                                        | 关键可变属性               |
|---------------------------|-----------------------------------------------|----------------------|
| <b>Model artifact</b>     | <code>'model://llama/7b'</code>               | 权重、格式、来源、许可证         |
| <b>Model version</b>      | <code>'model://llama/7b@sha256:...'</code>    | tokenizer、量化、训练数据摘要  |
| <b>Adapter</b>            | <code>'adapter://tenant-a/lora-v3'</code>     | base model、rank、发布日期 |
| <b>Endpoint</b>           | <code>'endpoint://prod-cn/chat'</code>        | 路由、灰度比例、SLO、当前版本     |
| <b>Serving deployment</b> | <code>'deploy://prod-cn/llm-server/v1'</code> | Pod 数、GPU、镜像、配置      |

关键区分 Endpoint 是面向调用者的契约；Deployment 是运行实现；Model version 是可复现的资产。三者必须分开建档。

## 数据与知识对象：Dataset、Vector Index、Prompt



“知识库”不是一串文档名：需要记录切分、embedding 模型、索引版本和生效范围。

---

## Agent Native 对象：Tool、Agent、Workflow、Evaluation

**Tool** 有输入/输出 schema 的外部能力。

**Agent** 策略、模型、记忆和工具集的组合。

**Workflow** 有状态、依赖、审批与终态的流程。

**Evaluation** 数据集、裁判、阈值和版本化结果。

- ▶ Agent 的版本变化可能来自 prompt、tool schema、模型或 policy；
- ▶ 只记录 ‘agent=v3’ 不足以复现一次决策；
- ▶ 对象档案应保存组成关系和生效时间。

## 四种字段：身份、描述、观测、推断



- ▶ ‘pod.uid’ 是身份候选；‘team=platform’ 是描述；‘cpu=80%’ 是观测；
- ▶ “可能属于支付系统” 是推断，必须附 evidence 和 confidence；
- ▶ 后续自动化只应直接依赖身份和经验证的关系。



---

# 对象档案的最小结构

## 教学 schema

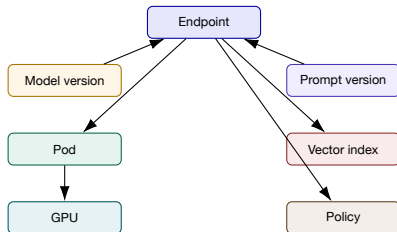
```
ObjectProfile = (id, kind, names, attributes, relations, provenance, confidence, valid_from, valid_to,
observed_at, state)
```

---

| 字段                    | 作用                                |
|-----------------------|-----------------------------------|
| <b>id/kind</b>        | 稳定身份与对象类型                         |
| <b>names</b>          | 官方名、短名、日志别名、历史名                   |
| <b>attributes</b>     | 版本、owner、region、配置等描述             |
| <b>provenance</b>     | 来源 URI、抽取器、原文片段、时间                |
| <b>confidence</b>     | 属性或关系的置信度，不是模型自信分数的替代品            |
| <b>valid/observed</b> | 世界中有效的时间与我们看到它的时间                 |
| <b>state</b>          | active、retired、unknown、tombstoned |

---

## 为什么 AI Native 需要更多对象类型？



**AI Native 特征** 一次响应的行为由模型版本、Prompt、检索索引、工具、策略和资源共同决定；只建“服务”节点会丢失可解释性。

## 稳定身份：为什么不能只用名字？

**名字会复用** Pod、Job、版本标签和 endpoint alias 都可能重用。

**名字会漂移** 服务改名、迁移命名空间、团队重组。

**名字会冲突** 多集群、多租户、多区域中出现同名对象。

- ▶ Kubernetes Pod：优先使用 namespace + UID；
- ▶ Process：使用 pid + start time；
- ▶ Model：使用 registry URI + immutable digest；
- ▶ Endpoint：使用环境、租户、服务契约和版本历史的组合键。

来源：OpenTelemetry Entities；Kubernetes Object UID；OpenLineage Dataset namespace/name。

---

## 教学用 Canonical ID：用 URI 让关系可读

### 示例

```
host://prod-cn/h-17  
pod://prod-cn/ns-ai/llm-server-7d9f@uid-8f2  
model://registry/llama-7b@sha256:abc...  
endpoint://prod-cn/tenant-a/chat@v3
```

- ▶ URI 不是标准答案，而是把 scope、kind、自然键和不可变版本显式化；
- ▶ 同一个实体的短名可以保存在 aliases 中，但不能替代 canonical ID；
- ▶ ID 设计一旦改变，关系图、审计、历史和训练数据都会受到影响。

## 对象生命周期：创建、变化、心跳、删除



丢失或乱序时  
需要 TTL / expiration

对象的删除事件可能丢失；“最后一次看到”不能直接等价于“对象仍然存在”。

来源：OpenTelemetry Entity Events, Development specification。

---

## 小结：对象宇宙三条设计原则

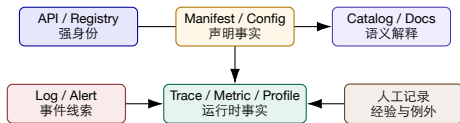
1. 先分辨对象，再提取属性：metric、log、profile 是证据，不是万能实体；
2. 先设计身份，再设计关系：无稳定 ID，图上的边只能是字符串猜测；
3. 先保存来源和时间，再允许自动化：不透明的推断不应进入生产决策。

**过渡** 下一节从“对象应该是什么”转向“如何从配置、文档和观测中把它抽取出来”。

# 三、多源抽取：从材料到候选对象

Schema-first、来源分级和可复核的模型抽取

## 多源材料：每一种来源都只看到世界的一部分



来源分级 声明配置通常强于自由文本；运行时 telemetry 能确认“曾经发生”，但不一定能证明“应该是什么”。

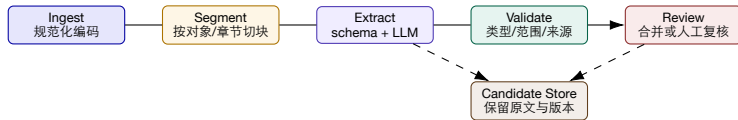


# Schema-first: 先规定 “要抽什么”

|                            |                                        |                          |
|----------------------------|----------------------------------------|--------------------------|
| 字段                         | Prompt / parser 输出                     | 校验                       |
| <b>object.kind</b>         | 'Deployment'、'Endpoint'、'ModelVersion' | 枚举 + scope               |
| <b>canonical_candidate</b> | 名称、namespace、digest                    | URI 格式 + registry lookup |
| <b>attribute</b>           | owner、region、version、GPU               | 类型、范围、来源                 |
| <b>relation</b>            | serves、runs_on、uses                    | 端点类型兼容矩阵                 |
| <b>evidence</b>            | 文档 URI + 片段 + 行号                       | 可回放、可定位                  |
| <b>uncertainty</b>         | unknown / conflict / candidate         | 不允许静默填默认值                |

Schema 不只是输出格式，它把 “模型会不会胡乱创造字段” 变成可以检查的工程问题。

## 一条可复核的抽取流水线



不要跳过 Segment、validate、review 三步决定了抽取结果能否被另一位工程师复核。

---

# 本地案例的声明配置：一个 **Deployment** 暴露了什么？

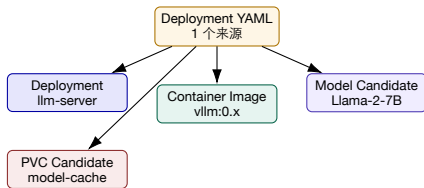
## 来自教学仓库的清单片段

```
metadata.name: llm-server labels.app: llm-server component: inference
nodeSelector: kubernetes.io/role=gpu-node image: vllm/... model_name: meta-llama/Llama-2-7b-chat-hf
resources.limits.nvidia.com/gpu: 1 PVC: llm-model-cache port: 8000
```

- ▶ 直接可得：Deployment、容器镜像、模型名称、GPU 约束、PVC、端口；
- ▶ 需要补证据：真正运行的 Pod UID、GPU device、实际 model digest；
- ▶ 不应猜测：owner、SLO、灰度比例、数据驻留策略。

来源：本地教学材料：ai-infra-engineer-learning-main / project-103-llm-deployment。

## 从清单到候选对象：不要把一个文件当成一个实体



同一起来源可产生多个对象和多条关系；抽取器应输出证据指针，而不是只输出一个摘要。

## SIGMOD 教程的启发：半结构化数据要先结构化

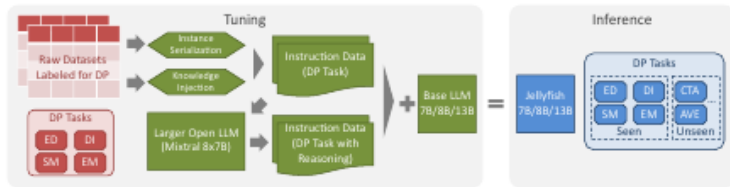
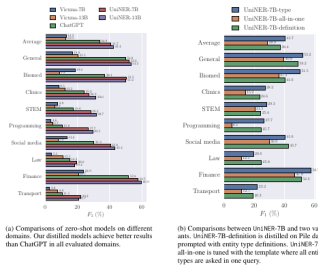


Figure 1: Overview of instruction tuning for data preprocessing.

- ▶ 原始文档、表格和配置并不共享一个天然 schema；
- ▶ 离线抽取、指令微调 and 推理阶段的 schema 约束可降低后处理成本；
- ▶ 课堂迁移：对象档案必须把“抽取—校验—合并”分成可测量阶段。

来源：Jellyfish: A Large Language Model for Data Preprocessing, Fig.1 独立裁切；SIGMOD 2025 Data+AI tutorial 的结构化抽取主题。

# UniversalNER: 开放实体抽取能覆盖长尾词汇



- 任务：在预先未知的实体类型上做开放 NER；
- 论文在 43 个数据集、9 个领域评估，UniNER 的平均结果显著高于零样本基线；
- 对运维的启发：文档中的“模型别名、服务角色、资源用途”可以先由 LLM 发现。

来源：UniversalNER, ICLR 2024, Fig.3 独立裁切；图中数值为论文实验结果。

## UniversalNER 的结果应该如何解读？

**43**

数据集

**9**

领域

**+30**

相对 Alpaca/Vicuna 的平均 F1 个  
百分点以上

- ▶ 论文还报告平均优于 ChatGPT 约 7-9 个百分点；
- ▶ 这是“抽取边界识别”的结果，不是“Canonical ID 命中率”；
- ▶ 运维对象有 namespace、版本、租户、生命周期等额外约束，必须接入 registry 和规则校验。

来源：UniversalNER, ICLR 2024；结果依赖数据集、模型和提示设置。

## 把 UniversalNER 迁移到运维：一条安全链



工程结论 LLM 负责“发现候选”，registry、类型约束和证据负责“确认对象”。



## Jellyfish: 把抽取难题推进到 matching

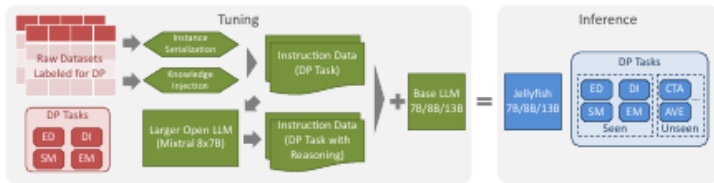


Figure 1: Overview of instruction tuning for data preprocessing.

- ▶ 论文覆盖 error detection、imputation、schema matching、entity matching；
- ▶ 这些任务更接近“同一个服务的不同写法是否对应同一对象”；
- ▶ 本讲重点使用 schema/entity matching，而不是把所有数据预处理任务都展开。

来源：Jellyfish, arXiv:2312.01678, Fig.1 独立裁切。

# Jellyfish 的结果：本地模型也可以成为 verifier

| 任务                              | 论文结果                             | 课堂解释             |
|---------------------------------|----------------------------------|------------------|
| 平均数据预处理                         | Jellyfish-13B 86.02; GPT-4 84.17 | 专用调优模型可接近或超过通用模型 |
| 未见过的 <b>CMS schema matching</b> | Jellyfish-13B 59.26; GPT-4 19.35 | 泛化到新 schema 仍然困难 |
| <b>Synthea schema matching</b>  | Jellyfish-13B 56.0; GPT-4 66.67  | 不同数据集上优势会反转      |
| <b>Abt-Buy entity matching</b>  | Jellyfish-13B 89.58; GPT-4 92.77 | 候选生成仍是关键瓶颈       |

边界 论文数字说明模型在特定任务/数据集上的效果，不能直接替换生产对象目录。

来源：Jellyfish, arXiv:2312.01678；表 4、表 5 和 entity matching 结果。

# 抽取与对齐的区别

## 抽取 Extraction

- ▶ 文本中有没有 “llm-server”?
- ▶ 它可能是什么类型?
- ▶ 哪些属性出现在同一段落?

## 对齐 Entity Resolution

- ▶ ‘llm-server’ 是否等于 ‘model-gateway’?
- ▶ 它属于哪个 namespace / tenant?
- ▶ 与已有对象的合并是否会造成错误影响?

**常见误区** 抽取 F1 高，不代表同名对象不会被错误合并。

## 多源冲突：同一字段为什么会有两个答案？

| 属性                   | Manifest  | Telemetry / 文档         | 处理策略                  |
|----------------------|-----------|------------------------|-----------------------|
| <b>model.version</b> | 'v3'      | digest 'abc...'        | digest 优先；v3 留作 alias |
| <b>owner</b>         | 'team-a'  | runbook 写 'team-b'     | 标记 conflict，进入复核      |
| <b>region</b>        | 'prod-cn' | trace resource=unknown | 不用 unknown 覆盖声明事实     |
| <b>service.name</b>  | 缺省        | OTel=unknown_service   | 提示接入方显式设置             |

冲突原则 保留多份证据和观察时间，不把“最后到达”误当成“最高可信”。

## 抽取质量：至少要测五个维度

**P**

实体/属性 precision

**R**

实体/属性 recall

**F1**

对齐质量

**U**

unresolved rate

**C**

provenance  
coverage

- ▶ P/R/F1 测“抽取和对齐是否正确”；
- ▶ unresolved rate 测系统是否诚实地承认未知；
- ▶ provenance coverage 测每个字段能否回到原文或 telemetry；
- ▶ 还需记录人工复核耗时、冲突率和新 schema 的退化幅度。

---

## 本节小结：LLM 是候选生成器，不是最终目录

1. Schema-first 让输出可以验证；
2. UniversalNER 说明长尾实体发现有价值；
3. Jellyfish 说明 matching 才是更难的生产问题；
4. 冲突、未知和无来源字段必须显式保留；
5. 进入下一节：如何把候选对象合并为一个稳定、可回放的档案。

## 四、实体对齐与对象档案

从“看起来相似”到“有证据地确认同一对象”

# 实体对齐的基本问题

给定

候选提及  $m$ 、已有对象  $o$ 、多源证据  $E$ ，判断  $m$  是否指向  $o$ ，或者创建新对象。



决策必须支持三种结果：合并、创建新对象、暂不确定。只允许二分类会强迫系统犯错。



## Blocking: 不要让每个提及与全目录比较



- ▶ Blocking key 可以是 namespace、租户、region、kind、digest 前缀；
- ▶ 召回优先：候选少但漏掉真实对象，后续 verifier 无法挽回；
- ▶ 生产系统要记录候选生成器和版本，便于重放。

---

## 匹配信号：规则、向量、LLM 各自负责什么？

---

|              |                             |             |
|--------------|-----------------------------|-------------|
| 信号           | 擅长                          | 风险          |
| 唯一键/UID      | 精确确认同一对象                    | 资料缺失时无法工作   |
| 规范化字符串       | 处理大小写、连字符、短名                | 容易把同名对象混在一起 |
| 结构化属性        | namespace、region、owner、port | 配置过期或不完整    |
| 向量相似度        | 语义相近、长尾别名                   | 相似不等于同一     |
| LLM verifier | 解释冲突、阅读证据                   | 成本、幻觉、不可重复  |

---

组合策略 规则负责硬约束，向量负责召回，LLM 负责解释与排序，人工负责高风险冲突。

---

## 一个可教学实现的 Match Score

### 示意评分

$$S(m, o) = 0.35 I_{uid} + 0.20 I_{scope} + 0.20 \text{sim}(\text{name}) + 0.15 I_{version} + 0.10 \text{sim}(\text{context})$$

- ▶  $I_{uid}$ 、 $I_{scope}$ 、 $I_{version}$  是可解释的离散信号；
- ▶ ‘sim(context)’ 可以由 embedding 或 LLM 计算，但必须保存输入；
- ▶ 阈值不是普适常数：按对象类型、风险和误合并成本分别校准。

---

## 匹配决策：把阈值和风险绑定

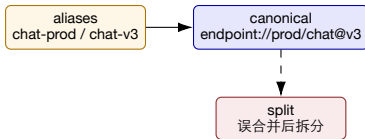
$S \geq 0.95$  低风险对象可自动合并，但保留审计记录。

$0.70 \leq S < 0.95$  进入 LLM verifier 或人工复核。

$S < 0.70$  创建候选新对象或保持 unresolved。

**高风险例外** 模型版本、租户、生产 endpoint 和权限对象宁可 unresolved，也不要错误合并。

## 合并、拆分与别名表



- ▶ 别名是可变的，canonical ID 不能随别名变化；
- ▶ merge 需要记录合并原因和生效时间；
- ▶ split 需要保留旧对象与新对象的 lineage，不能直接删除历史。

# 对象档案：把属性变成“有来源的事实”

| 属性           | 值           | 证据与时间                                         |
|--------------|-------------|-----------------------------------------------|
| kind         | Endpoint    | registry API, observed=2026-08-21T09:00       |
| version      | v3          | deployment label, valid=2026-08-20..          |
| model_digest | sha256:abc  | model registry, immutable                     |
| owner        | platform-ai | catalog + runbook, conflict=false             |
| gpu_count    | 1           | manifest declaration, runtime observation 待确认 |

关键原则 同一个字段允许有多个版本化事实；“当前值”只是查询视图，不是唯一存储。

# 来源、置信度与模型输出要分开

## 来源 Provenance

- ▶ 文件 URI
- ▶ API resource version
- ▶ trace/profile ID
- ▶ 原文片段

## 置信度 Confidence

- ▶ 抽取器置信度
- ▶ 规则校验结果
- ▶ 人工确认
- ▶ 冲突状态

## 决策 Decision

- ▶ 入档
- ▶ 保留候选
- ▶ 标记 unknown
- ▶ 触发复核

模型给出的概率不能直接代表业务事实的可信程度；来源质量、时间新鲜度和冲突状态也必须进入决策。

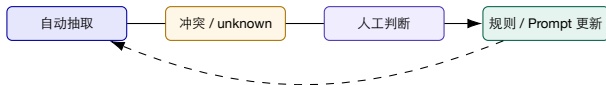
## 双时间：世界何时有效，我们何时看到？



- ▶ 2026-08-20 发布的模型可能在 08-21 才被 telemetry 观察到；
- ▶ 事件发生时间和日志到达时间不同，关系图也会遇到乱序；
- ▶ 没有双时间，历史查询会把“后来知道的事实”错误投射回过去。



## 人工复核不是失败，而是对象图的主动传感器



- ▶ 复核结果应落为 relation/attribute 的 evidence，而不是只留在聊天记录；
- ▶ 高风险对象优先获得人工预算；
- ▶ 复核数据可用于改进 blocking key、schema 和 verifier。

---

## 隐私与最小必要信息

**提示词** 可能包含客户、代码、内部拓扑。

**日志** 请求内容、用户 ID、token 片段。

**人工记录** 工单、值班人和事故细节。

- ▶ 对象档案保存“可追溯证据”，不等于保存全部原文；
- ▶ 用脱敏片段、哈希、访问控制和保留期限降低暴露面；
- ▶ provenance URI 要支持权限检查，不能把敏感日志复制进公共知识库。

---

## 本节结论：高质量档案允许“不确定”存在

1. Blocking 控制候选规模，避免全量两两比较；
2. 规则、向量、LLM 和人工形成分层决策；
3. merge/split、alias、双时间和 provenance 让档案可回放；
4. unresolved 是一种合法状态，不是要被模型强行填满的空白。

## 五、从对象档案到关系图

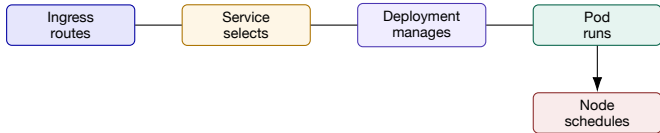
把“谁是谁”推进到“谁依赖谁、谁产生谁、谁观察谁”

# 关系的六个来源



不同来源的关系语义不同；不要把“被同一段日志提到”当成“依赖”。

## 声明关系：配置文件中写下来的边



- ▶ 这些边可由 manifest、API 和 ownerReferences 生成；
- ▶ 需要把 namespace、UID、resourceVersion 和采集时间保存为证据；
- ▶ 声明关系回答“应该怎样”，不保证某一时刻真的这样运行。

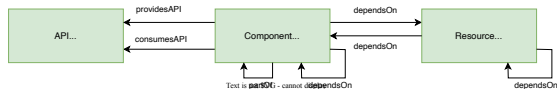
# Kubernetes: ownerReferences 与 labels/selectors 不是一回事

| 机制                     | 语义               | 典型误用                        |
|------------------------|------------------|-----------------------------|
| <b>ownerReferences</b> | 控制器所有权、级联删除和依赖方向 | 把 owner 当成所有业务依赖            |
| <b>labels</b>          | 用户定义的识别/组织属性     | 把 label 当成唯一 ID             |
| <b>selectors</b>       | 选择一组对象的分组原语      | 把 selector 命中的全部 Pod 当成静态成员 |
| <b>UID</b>             | 对象实例的不可变身份       | 只用 name 追踪重建后的 Pod          |

工程结论 控制面关系、选择关系和业务依赖关系应使用不同 edge type。

来源：Kubernetes 官方 owners/dependents 与 labels 文档。

## Backstage: 软件目录里的语义关系

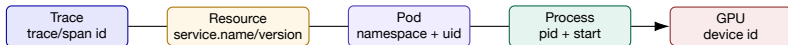


- ▶ Component 可以实现或消费 API，并依赖 Resource；
- ▶ System/Domain 让对象图可以按团队、业务和系统浏览；
- ▶ 课堂迁移：把 endpoint、model service、vector index 作为目录对象，再接到运行时实体。

来源：Backstage Software Catalog System Model，官方图。



## 运行时关系：由 Resource 与 Entity 连接 telemetry



- ▶ OTel Resource 描述产生 telemetry 的实体；
- ▶ `service.name`、`service.version`、`deployment environment` 是重要 join key；
- ▶ Resource 缺省为 `unknown_service` 时，后续图合并会失去入口。

来源：OpenTelemetry Resources，官方文档，2026-02 更新。

# OpenTelemetry Entity: 最小身份与可变描述

## Entity ID

- ▶ 至少一个 identifying attribute;
- ▶ 类型和 ID map 不应随描述变化;
- ▶ 例如 process = pid + start time.

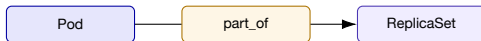
## Description attributes

- ▶ owner、region、image、版本等可变信息;
- ▶ 变更需要 state event 或新观察;
- ▶ 不应把描述属性拼入不稳定的主键。

设计启发 对象图的身份层要足够小，描述层要足够丰富；二者混在一起会导致重复对象。

来源: OpenTelemetry Entities specification, Development。

## Entity Events: 关系也需要生命周期



关系边包含 type、observed\_at、valid\_to、evidence

- ▶ 高 churn 的短生命周期对象更适合携带关系，例如 Pod → part\_of → ReplicaSet；
- ▶ 删除事件可能丢失，因此 receiver 要根据报告间隔设置 TTL；
- ▶ relation type 不能只写 related\_to，否则无法支持下游查询。

来源：OpenTelemetry Entity Events, Development。

# iText2KG: 增量构建图，而不是一次性生成整图

mance. Further details of modules 1 to 3 are as follows, with the fourth module serving to visualize the graph.

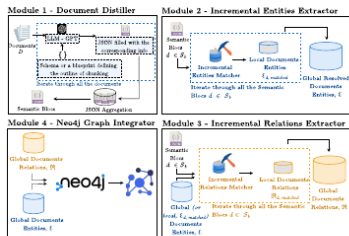


Fig. 1. The overall workflow of the iText2KG modules. Module 3, the Incremental Relations Extractor, operates differently depending on whether global or local document

- ▶ Document Distiller 将长文档切为语义块；
- ▶ Incremental Entity/Relation Extractor 只处理局部变化；
- ▶ Graph Integrator 负责去重、合并和可视化。

来源：iText2KG, arXiv:2409.03284, Fig.1 独立裁切。

## iText2KG 的结果：局部上下文更精确，全局上下文更丰富

**0.94**

CS 局部上下文 triplet precision

**0.83**

CS 全局上下文 triplet precision

- ▶ 音乐数据集同样呈现局部约 0.90、全局约 0.81 的趋势；
- ▶ 全局上下文可以发现跨文档关系，但会引入约 10 个百分点的精度损失；
- ▶ 运维迁移：先用对象局部上下文抽取，再在有证据时扩展跨系统关系。

来源：iText2KG，表 2；论文中的小规模评估，不等同生产准确率。

# iText2KG 的图对比：为什么需要增量整合？

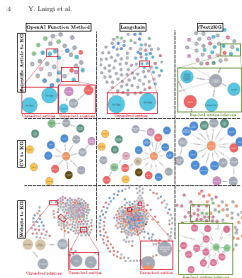


Fig. 5. Comparison of KG construction across three scenarios between baseline methods and our method, iText2KG.

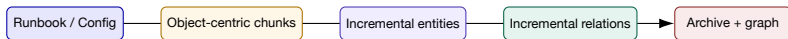
breed limitations inherent in traditional KG construction processes, which typically depend on predefined ontologies and extensive supervised training.

A key advantage of the iText2KG approach is its flexibility, which stems

独立裁切展示三类输入场景下的未解析实体/关系与已解析关系；课堂关注“整合和消歧”的作用，不把图形大小当成精确指标。

来源：iText2KG, Fig.5 独立裁切。

## 把 iText2KG 迁移到运维对象图



- ▶ 文档更新只重算受影响的对象块；
- ▶ 每条边携带 **source chunk**、**extractor version** 和 **observed time**；
- ▶ 图整合器负责别名、冲突和时间窗口，而不是让 LLM 自由合并。

## 关系类型设计：一组最小但可查询的词表

| 关系                                  | 示例                                                         | 查询用途       |
|-------------------------------------|------------------------------------------------------------|------------|
| <b>contains / part_of</b>           | Pod part_of ReplicaSet                                     | 生命周期与控制器追踪 |
| <b>routes_to / exposes</b>          | Ingress routes_to Service                                  | 入口到服务路径    |
| <b>runs_on / scheduled_on</b>       | Process runs_on GPU                                        | 运行时归因      |
| <b>serves / implements</b>          | Endpoint serves ModelVersion                               | 模型版本影响面    |
| <b>uses / indexed_by</b>            | Agent uses Tool; RAG uses Index                            | AI 行为组成    |
| <b>derived_from / documented_by</b> | Profile derived_from Process; Object documented_by Runbook | 证据与血缘      |

命名原则 边的方向、类型、有效时间和证据必须明确；`related_to` 只能作为临时未知边。



---

## 关系置信度：边也要有证据等级

**A**

API / UID 直接确认

**B**

配置或控制面声明

**C**

Telemetry / 文档推断

- ▶ A 级边可直接供自动化查询；
- ▶ B 级边要标出“期望关系”，与运行时事实区分；
- ▶ C 级边需要 evidence、confidence 和过期时间，不能自动触发高风险动作。

---

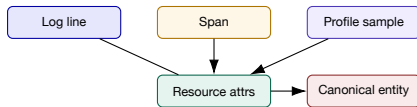
## 本节小结：关系图不是一张漂亮的网络图

1. 关系类型表达语义，来源表达证据，时间表达生命周期；
2. Kubernetes、Backstage、OTel、OpenLineage 提供互补的对象模型；
3. iText2KG 提醒我们：增量、局部上下文和图整合比一次性生成更可靠；
4. 下一节把 telemetry 和 profiling 作为“运行时关系传感器”。

## 六、可观测性与 Profiling: 关系图的运行时传感器

不重讲指标算法，而是回答“观测如何归属到对象和版本”

## 观测数据的主问题：它到底属于谁？



- ▶ 统一 join key: service.name、service.version、namespace、pod UID、process start;
- ▶ 观测没有归属就只能做全局统计，无法回写对象历史；
- ▶ 归属关系与 RCA 不同：本讲只把观测挂到对象上。

---

# OTel Resource：对象档案的运行时入口

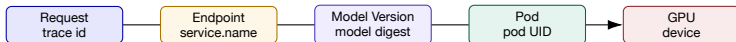
## 典型 Resource

```
service.name=llm-gateway service.version=v3 deployment.environment=prod  
k8s.namespace.name=ai k8s.pod.uid=... host.name=gpu-node-17
```

- ▶ Resource detector 可从 OS、host、process、container、Kubernetes 和云环境发现属性；
- ▶ 显式设置 `service.name`，避免默认的 `unknown_service`；
- ▶ Resource 属性是 join key，不是把所有业务标签塞进每一条 span。

来源：OpenTelemetry Resources 官方文档。

## AI 服务的 trace 关联：请求—端点—模型版本



- ▶ 这条链可以回答“这次请求使用了哪个版本、落在哪个 Pod/GPU”；
- ▶ Prompt、检索索引、tool call 也可作为 span/link 的对象引用；
- ▶ 只有归属关系时，才有可能在后续章节计算对象级事件历史。

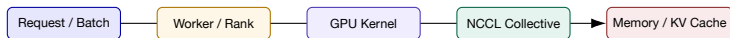
## Profiling: 从服务归属到进程和代码路径



- ▶ OTEL Profiles 目标：低开销、连续生产 profiling，与 logs/metrics/traces 关联；
- ▶ Profile 由 ResourceProfiles、ScopeProfiles、Profile、Sample 等组成；
- ▶ 样本可以链接到 trace/span，从而把代码路径挂到一次具体请求。

来源：OpenTelemetry Profiles specification, Alpha。

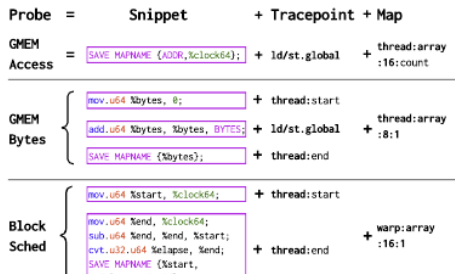
## AI Native profiling 的新对象：Kernel、Collective、Batch



- ▶ GPU 异步执行、批处理和分布式 collective 让“一个慢点”难以归因；
- ▶ 对象图要把 worker、rank、kernel、batch 和模型版本分开；
- ▶ 本讲只建立归属与关系，不展开 GPU 调度或性能优化算法。



# OSDI 2025 Neutrino: 把 GPU kernel 内部变成可查询对象



- ▶ probe = snippet + tracepoint + structured map;
- ▶ assembly-layer probing 覆盖 NVIDIA/AMD;
- ▶ 可记录地址、字节、设备时钟与 block scheduling;
- ▶ cooperative probes 连接多个 tracepoint;
- ▶ 产物应关联 kernel、rank、model version 与 profile ID。

来源: Neutrino: Fine-grained GPU Kernel Profiling via Programmable Probing, OSDI 2025, Fig. 4。

## Neutrino 的结果与对象图落点

**1.04×**

多数轻量 probe 的 kernel  
slowdown

**+4.11**

平均额外 registers

**~28%**

block scheduling 案例 speedup

- ▶ 案例发现约 20% 时间用于 block 调度到物理 SM;
- ▶ DMAT 将 memory address、time、thread/block 层次转成可比较 profile;
- ▶ 对象图需新增 probe config → kernel image → sample map;
- ▶ 案例 speedup 不可外推; instruction-level probe 不是默认连续观测手段。

来源: Neutrino, OSDI 2025, §5–§6。

---

## Coz 的思想: profiling 可以问 “优化它会带来什么?”

**+9%**

Memcached

**+25%**

SQLite

**+68%**

部分 PARSEC 应用最高

- ▶ Coz 用虚拟加速估计某段代码对端到端吞吐的因果影响;
- ▶ 启发: profile 不只是 “哪里最热”, 还可以连接到 “哪条边影响了对象结果”;
- ▶ 边界: CPU 程序上的因果 profiling 不能直接覆盖 GPU 异步、批调度和跨节点通信。

来源: Coz: Finding Code that Counts with Causal Profiling, SOSP 2015 Best Paper。

## 把 profiling 接入对象图，而不是另建孤岛



每条边保存 observed\_at、source、confidence 和 expiry

本讲的“可观测性” 重点是 attribution 与 lineage：谁产生了这条样本、样本对应哪个版本，而不是重新讲聚合、采样和告警阈值。

## 对象图中的“声明世界”和“运行世界”

### 声明世界

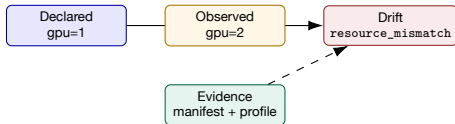
- ▶ Deployment 期望 1 个 GPU;
- ▶ nodeSelector 指向 gpu-node;
- ▶ model\_name 为 Llama-2-7B.

### 运行世界

- ▶ Pod 实际落在 node-17;
- ▶ trace 的 model digest 为 abc;
- ▶ profile 观察到 2 个 GPU process。

**不要覆盖** 声明与运行事实可以不一致；图数据库应保留两类边，并用 reconciliation 生成差异。

## 关系重 conciliation: 差异本身也是对象证据



- ▶ Drift 不是直接的故障结论，而是需要记录的关系差异；
- ▶ 差异可以由策略、扩容、采样缺失或错误标签造成；
- ▶ 后续事件分析可消费 drift，但不在本讲展开告警合并。

---

## 可观测性接入的最小字段集

| 字段                           | 示例                    | 作用         |
|------------------------------|-----------------------|------------|
| <b>service.name/version</b>  | llm-gateway / v3      | 服务与版本归属    |
| <b>k8s.namespace/pod.uid</b> | ai / uid-8f2          | 运行时实体身份    |
| <b>model.name/digest</b>     | llama-7b / sha256:abc | 模型 lineage |
| <b>trace.id/span.id</b>      | 16/32 字节 ID           | 请求级关联      |
| <b>tenant / endpoint</b>     | tenant-a / chat       | 范围和契约      |
| <b>source / observed_at</b>  | otel / timestamp      | 来源与时间      |

---

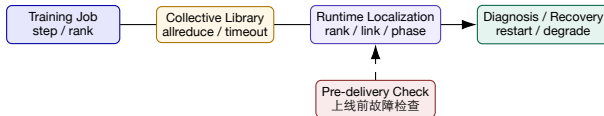
字段越少越容易落地，但缺少 model digest、Pod UID 或版本会让 AI Native 关系无法复现。

## 七、近期 AI 集群论文专题

把 collective、网络路径、GPU 利用率和观测成本纳入同一条证据链



## NSDI 2025 Aegis: 在 collective 内做运行时故障定位



- ▶ Aegis 从通用诊断增强，演进到定制 collective library；
- ▶ 不修改客户代码，就能在运行时定位通信/训练故障，并覆盖性能退化和交付前检查；
- ▶ 教学重点：把 step、rank、collective、通信阶段纳入对象和关系，而不是只记录主机级指标。

来源：Evolution of Aegis: Fault Diagnosis for AI Model Training Service in Production, NSDI 2025。

## Aegis 的生产结果：诊断信号必须能改变恢复路径

**>97%**

诊断造成的 idle time 降低

**84%**

训练任务 restart 次数降低

**71%**

性能退化降低

- ▶ 官方摘要称 Aegis 已在生产训练云部署一年；
- ▶ 结果不是单一模型预测带来的，而是诊断、定位、恢复和交付前检查的组合；
- ▶ 第 15 讲只抽取它对对象图的启发：job → step → rank → collective → network path。

来源：Aegis, NSDI 2025；数字来自官方会议摘要。

## OSDI 2025 What-if: 为训练 Step/Rank 关系增加反事实权重

- ▶ 对可比较 worker/operation 构造无 straggler 的反事实时间线；
- ▶ 分别“修复”compute、PP/DP communication、stage 或 worker，估计可恢复 JCT；
- ▶ 五个月 3,079 作业中，42.5% 至少慢 10%，总体浪费 10.4% allocated GPU-hours；
- ▶ 仅 1.7% straggling jobs 的多数 slowdown 可由少数慢 worker 解释；
- ▶ 对象图落点：actual duration、counterfactual duration、recoverable slowdown 与估计方法版本。

**边界** 论文集群网络供给充分；counterfactual 是模型估计，不应覆盖原始 trace。

来源：Understanding Stragglers in Large Model Training Using What-if Analysis, OSDI 2025。

## NSDI 2026 FLARE: 全栈训练 Trace 怎样归属到对象?

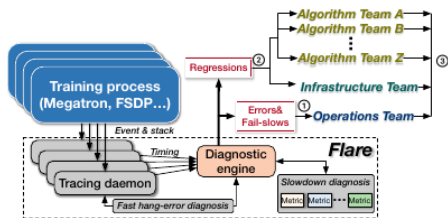


Figure 2: Architecture overview of FLARE.

- ▶ daemon 拦截 Python/C++ 与关键 GPU kernel;
- ▶ stack reconstruction 连接异步 kernel 与上层调用;
- ▶ issue latency、void percentage、FLOPS、communication bandwidth 形成派生对象证据;
- ▶ 结果路由到 algorithm/infrastructure/operations owner。

来源: FLARE: Anomaly Diagnostics for Divergent LLM Training in GPU Clusters of Thousand-Plus Scale, NSDI 2026, Fig. 2。

## FLARE 结果：低开销来自选择性观测与对象对齐

**0.43%**

三种 LLM backend 的延迟开销

**81.8%**

一周生产回归 true-positive  
diagnostic accuracy

**6,000**

GPU 部署，连续 8+ 月

- ▶ 1536 H800 作业每 GPU 约 1.5MB trace；生产回归 false-positive rate 1.9%；
- ▶ 成本不是“采集全部细节”，而是只拦截关键 API/kernel 并保存 stack/rank/input shape；
- ▶ 对象图必须保存 instrumentation/version/coverage，否则“未见异常”可能只是未覆盖；
- ▶ 第 17 讲将把这些字段作为 Agent 的高成本下钻工具，而非本讲直接做 RCA。

来源：FLARE, NSDI 2026, §6 and production deployment。

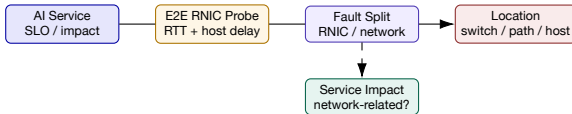
# NSDI 2024: MegaScale 与 TPUv4 的两种稳定性视角

|                     |                                            |                                                          |
|---------------------|--------------------------------------------|----------------------------------------------------------|
| 工作                  | 观测/定位方法                                    | 公开结果                                                     |
| MegaScale           | 深入监控系统组件和事件，诊断 failure/straggler，形成容错与缓解技术 | 175B 模型、12,288 GPUs；55.2% MFU；较 Megatron-LM 1.34×        |
| Resiliency at Scale | TPU ICI fabric 的故障检测、动态路由重配置、维修 workflow   | 4096-node TPUv4；99.98% availability；约 1% 训练任务经历硬件 outage |

共同启发 大规模训练需要同时观察模型 step、通信阶段、设备/链路和维护状态；只保留 endpoint 指标无法解释 straggler。

来源：MegaScale；Resiliency at Scale，NSDI 2024 官方摘要。

# SIGCOMM 2024 R-Pingmesh: 服务感知的 RoCE 诊断



- ▶ 端到端 probing 同时估计网络 RTT 与端主机处理延迟；
- ▶ 区分 RNIC 丢包与网络内丢包，判断问题是否真正影响服务；
- ▶ 官方摘要：数万 RNIC 部署超过 6 个月；一个月评估中 85% 定位准确，157 个交换机网络问题全部准确，覆盖 14 类问题。

来源：R-Pingmesh: A Service-Aware RoCE Network Monitoring and Diagnostic System, SIGCOMM 2024。

# SIGCOMM 2024 Crux/HPN: 通信竞争和拓扑也要成为对象关系

## Crux

- ▶ 生产 DLT 中跨任务通信竞争降低 GPU 利用率;
- ▶ GPU-intensity-aware scheduling;
- ▶ 96-GPU 测试床提升 8.3%–14.8%，仿真最高 23%。

## HPN

- ▶ LLM 训练流量周期性、突发且同步;
- ▶ 2-tier dual-plane、dual-ToR，降低 ECMP hash polarization 与 ToR 单点故障风险;
- ▶ 生产部署超过八个月。

对象图落点 job → flow → collective → path → ToR/RNIC → GPU utilization; 通信竞争和拓扑变化是可观测关系，不是单机异常。

来源：Crux; Alibaba HPN, SIGCOMM 2024。



---

## 专题结论：把论文方法翻译成对象图字段

| 论文方法                                    | 需要新增的对象/关系                                   | 后续可以回答的问题          |
|-----------------------------------------|----------------------------------------------|--------------------|
| <b>Aegis / MegaScale</b>                | Step、Rank、Collective、Straggler、Failure event | 哪个通信阶段拖慢了哪些 rank?  |
| <b>R-Pingmesh</b>                       | RNIC、Probe、Path、Host delay、Network impact    | 问题是否属于网络？影响哪个服务？   |
| <b>Crux / HPN</b>                       | Flow、GPU intensity、ToR、ECMP path、contention  | 利用率下降来自通信竞争还是资源故障？ |
| <b><math>\mu</math>View (NSDI 2026)</b> | Sketch、sampling budget、fine-grain metric     | 如何在观测成本受限时保留高信息信号？ |

这些论文为第 15 讲补充“关系和证据的来源”；故障排序、事件合并、自动恢复仍留给后续章节。

---

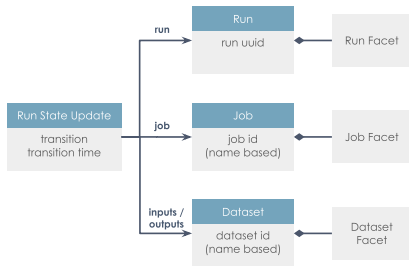
## 本节小结：Profiling 是“关系传感器”

1. Resource 把 telemetry 绑定到服务、版本、Pod 和主机；
2. Profile 把服务继续连接到进程、代码路径、GPU kernel 和 trace；
3. Coz 提醒我们关注因果影响，但 AI 系统需谨慎处理异步和分布式执行；
4. 这些关系先服务于对象档案和 lineage，诊断策略留给后续章节。

# 八、血缘与时间：让对象图 可以回放

模型、数据、Prompt、部署和观测必须在时间轴上对齐

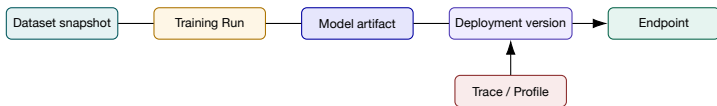
# OpenLineage: Job、Run、Dataset 的三层语义



- ▶ Job 是设计时的任务定义；Run 是一次执行；Dataset 是被读写的资产；
- ▶ Facets 承载 schema、版本、质量和运行时上下文；
- ▶ 课堂扩展：训练 Job → model artifact → deployment → endpoint。

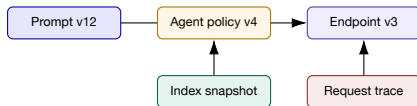
来源：OpenLineage Object Model 1.52.0，官方图。

# AI 模型血缘：从 Dataset 到 Endpoint



模型 artifact、部署版本和 endpoint alias 是不同对象；只有保留这条链，才能回答“某次请求到底使用了哪份数据和权重”。

## Prompt 与索引也要进入 lineage



- ▶ Prompt、policy、index 的变化会改变同一模型版本的行为；
- ▶ 在 profile 或 trace 中保存对象引用，而不是把完整敏感内容复制进去；
- ▶ lineage 先保证“可解释和可复现”，不等于自动判定质量。

---

## 时间一致性：三种常见错误

**未来穿越** 用后来采集到的版本覆盖过去的 endpoint。

**删除丢失** Pod 已消失但关系仍被无限期保留。

**乱序更新** 旧的观察晚到，覆盖了新事实。

- ▶ 采用 valid time + observed time；
- ▶ 使用 resourceVersion、sequence、event time 或 monotonic watermark；
- ▶ 对 heartbeat 关系设置 TTL，对 immutable digest 禁止回写。

---

## 图一致性检查：关系图也需要单元测试

---

|      |                                                 |
|------|-------------------------------------------------|
| 检查   | 示例断言                                            |
| 类型兼容 | Endpoint 'serves' ModelVersion; 不能反向连接 GPU      |
| 唯一性  | 一个 canonical ID 在同一 scope 只能对应一个 active profile |
| 时间   | relation.valid_from <= observed_at; 过期边不能参与当前查询 |
| 来源   | 每个高风险关系至少有 A/B 级来源或人工确认                         |
| 闭环   | Deployment → Pod → Node 的声明链不能出现悬空节点            |
| 漂移   | declared 与 observed 不一致时生成 drift evidence       |

---

工程结论 图一致性检查把“看起来合理”变成可自动回归的约束。



---

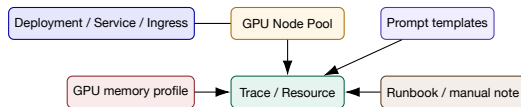
## 本节小结：血缘是对象图的时间轴

1. OpenLineage 提供 Job/Run/Dataset 的成熟语义；
2. AI Native 需要把 Model、Prompt、Index、Agent、Deployment 和 Endpoint 接入同一条 lineage；
3. 双时间、TTL、immutable digest 和图一致性测试让历史查询可信；
4. 下一节把前面的设计落到一个完整 LLM 部署案例。

# 九、综合案例：从 LLM Deployment 到对象档案

一套清单、一次观测和一组文档，如何合成可查询的 AI Native 对象图？

## 案例输入：一个 LLM 服务的材料包



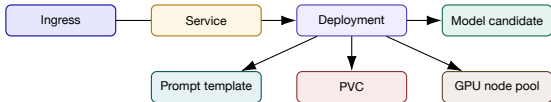
案例来自本地教学仓库中的 LLM deployment、GPU、prompt 和 observability 材料；数字不代表生产基线。

# Step 1: 生成对象清单

|                       |                                         |                                  |
|-----------------------|-----------------------------------------|----------------------------------|
| <b>kind</b>           | canonical candidate                     | 证据                               |
| <b>Deployment</b>     | 'deploy://prod/llm-server/v1'           | deployment.yaml name/labels      |
| <b>Service</b>        | 'service://prod/llm-server'             | service.yaml selector/port       |
| <b>Ingress</b>        | 'ingress://prod/llm-server-ingress'     | ingress.yaml backend             |
| <b>ModelVersion</b>   | model://hf/Llama-2-7b-chat-hf@candidate | model_name + registry lookup 待确认 |
| <b>PVC</b>            | 'pvc://prod/llm-model-cache'            | volume claim                     |
| <b>GPU Node Pool</b>  | 'pool://prod/gpu-node'                  | nodeSelector + device plugin     |
| <b>PromptTemplate</b> | 'prompt://chat/prod@candidate'          | templates.yaml                   |

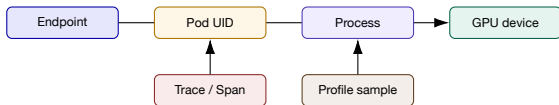
注意 'candidate' 表示尚未由 immutable digest 或 runtime observation 确认的对象，不应伪装成最终版本。

## Step 2: 生成声明关系



- ▶ `Ingress routes_to Service; Service selects Pod; Deployment uses Model;`
- ▶ `Deployment mounts PVC; Deployment scheduled_on Pool; Deployment uses Prompt;`
- ▶ 每条边写入 `source file`、`line/field`、`resourceVersion` 和 `valid_from`。

## Step 3: 用运行时观测补全实例关系



- ▶ Resource 'service.version' 与 model digest 让 endpoint 与 deployment 对齐;
- ▶ Pod UID、process start time 和 GPU device 解决实例级追踪;
- ▶ 若 runtime 没有 digest, 只能保留 'model candidate', 不能生成确定 lineage。

---

## Step 4: 输出可供后续章节消费的 Profile

### 对象档案摘要

```
id: endpoint://prod/llm-server/chat@v3
kind: Endpoint; state: active; owner: platform-ai
serves: model://hf/Llama-2-7b-chat-hf@sha256:abc...
runs_on: pod://prod/ai/llm-server-7d9f@uid-8f2
uses: prompt://chat/prod@v12; mounted: pvc://prod/llm-model-cache
provenance: manifest.yaml + otel.trace/abc + profile/xyz
valid_from: 2026-08-20; observed_at: 2026-08-21T09:00
```

下游接口 第 16 讲可以在此对象上追加事件历史；第 17 讲可以沿关系查询证据；第 14 讲可以将 endpoint 绑定到流程 Intent。

---

## 案例中的冲突与处理

| 冲突                        | 证据                 | 处理                                   |
|---------------------------|--------------------|--------------------------------------|
| 镜像使用 'latest'             | manifest           | 记录 mutable tag, 标记不可复现, 等待 digest    |
| Telemetry unknown_service | OTel Resource      | 不覆盖 manifest; 发出 instrumentation gap |
| Prompt 未版本化               | templates.yaml     | 生成 candidate, 要求补 hash/owner         |
| GPU 实际数与声明不符              | profile + manifest | 生成 drift evidence, 不自动判定故障           |
| owner 文档与目录不一致            | runbook + catalog  | 保留 conflict, 进入人工复核                  |

---

**原则** 把不一致变成结构化 evidence, 不能让模型替系统 “挑一个看起来合理的答案”。



---

## 案例评估：论文结果、规范事实、教学演示要分开

### 论文结果

UniversalNER、  
Jellyfish、iText2KG 的公  
开实验数字。

### 规范事实

OTel、  
Kubernetes、  
Backstage、  
OpenLineage 的官方语  
义。

### 教学演示

本案例中的  
对象、关系、阈值和示例  
冲突。

- ▶ 不把案例中的示意阈值写成论文结论；
- ▶ 不把规范中的字段存在性写成系统自动拥有该字段；
- ▶ 报告中必须注明数据集、采集时间、模型版本和人工修订。

---

## 课上练习：给一个新服务建立对象档案

1. 从 Deployment、Service、Ingress 和 prompt 文件中列出候选对象；
2. 为每个对象设计 canonical ID 与 aliases；
3. 画出至少 8 条有类型的声明关系；
4. 为两条运行时关系补上 Resource / trace / profile 证据；
5. 构造一个冲突，并写出 merge、review 或 unresolved 的理由；
6. 设计 5 个自动一致性检查。

交付物不是一张无来源的网络图，而是一份可回放的 object archive + relation manifest。

---

## 研究问题：对象图本身仍然值得研究

- ▶ 如何学习跨来源、跨版本的 canonical identity，而不把相似误当同一？
- ▶ 如何为 AI Agent、Prompt、Index 和模型版本定义可演化但稳定的对象模型？
- ▶ 如何在 telemetry 不完整、删除乱序和 schema 漂移时维护关系图？
- ▶ 如何评估“未知率降低”是否以错误合并率上升为代价？
- ▶ 如何把 profiling 的运行时证据与模型行为、数据血缘和业务结果对齐？

研究视角 对象档案不是静态 CMDB，而是一个带不确定性、时间和证据的增量知识系统。

---

## 本讲总结：五个带走的判断

1. AI Native 运维必须把模型、数据、Prompt、Agent 和 GPU 纳入对象宇宙；
2. 抽取与对齐是两个问题，LLM 候选生成不能替代身份确认；
3. 关系必须有类型、来源、置信度和有效时间；
4. OTel Resource/Profile、Kubernetes、Backstage、OpenLineage 提供互补的连接语义；
5. 一个诚实的 ‘unknown/unresolved’，比一个没有证据的“确定对象”更有用。

**下一讲** 在稳定对象图上解析告警和事件，构建对象级历史记忆；不重复本讲的身份与关系构建。

---

## 参考材料与阅读路径

1. UniversalNER: Targeted Distillation from Large Language Models for Open Named Entity Recognition, ICLR 2024。
2. Jellyfish: A Large Language Model for Data Preprocessing, arXiv:2312.01678。
3. iText2KG: Incremental Knowledge Graphs Construction Using Large Language Models, arXiv:2409.03284。
4. Coz: Finding Code that Counts with Causal Profiling, SOSP 2015。
5. OpenTelemetry Resources / Entities / Entity Events / Profiles 官方规范。
6. Kubernetes Owners and Dependents、Labels 官方文档。
7. Backstage Software Catalog System Model 官方文档。
8. OpenLineage Object Model 1.52.0 官方文档。
9. SIGMOD 2025 Data+AI tutorial: LLM4Data and Data4LLM。
10. 本地教学材料: ai-infra-engineer-learning-main 的 LLM deployment、GPU、Prompt 和 observability lessons。

图像使用独立裁切或官方图；论文结果均在页脚标注来源与适用边界。